

Replication Plan

DRAS: Deep Reinforcement Learning for Cluster Scheduling in High Performance Computing

OSTI ID: 1984484

Auto-generated Replication Blueprint

April 8, 2026

Contents

1	Paper Overview	2
2	Detailed Workflow	2
2.1	Phase 1: Environment Setup	2
2.2	Phase 2: Job Trace Data Preparation	2
2.3	Phase 3: State/Action/Reward Design	2
2.4	Phase 4: RL Agent Training	2
2.5	Phase 5: Evaluation and Comparison	3
3	Datasets	3
4	Models, Tools, and Software	3
4.1	Hardware Requirements	3

Paper Overview

DRAS applies deep reinforcement learning (DRL) to HPC cluster job scheduling. The authors develop CQGym, an OpenAI Gym-compatible environment wrapping CQSim (a cluster queue simulator), and train RL agents (DQL, Policy Gradient, A2C, PPO) to learn scheduling policies that optimize metrics like average bounded slowdown, average wait time, and system utilization. The RL agents are compared against traditional heuristics (FCFS, SJF, WFP, UNICEP) on real HPC job traces.

Detailed Workflow

Phase 1: Environment Setup

- 1.1 Install CQSim (cluster queue simulator).
- 1.2 Install/build CQGym (Gym wrapper around CQSim).
- 1.3 Install RL libraries: Stable-Baselines3 or custom implementations of DQL, PG, A2C, PPO.
- 1.4 Verify environment with a test episode.

Phase 2: Job Trace Data Preparation

- 2.1 Download HPC job traces.
 - Parallel Workloads Archive: ANL Intrepid, SDSC Blue, RICC, etc.
 - Convert to CQSim input format (Standard Workload Format).
- 2.2 Split traces into training and test periods.

Phase 3: State/Action/Reward Design

- 3.1 State representation.
 - Queue state: number of waiting jobs, resource requests, wait times.
 - System state: available nodes, running jobs, utilization.
 - Encode as fixed-size feature vector.
- 3.2 Action space.
 - Select which waiting job to schedule next (or select a scheduling heuristic from a portfolio).
- 3.3 Reward function.
 - Negative average bounded slowdown, or negative wait time, or utilization-based reward.

Phase 4: RL Agent Training

- 4.1 Train DQL, Policy Gradient, A2C, PPO agents on training traces.
- 4.2 Hyperparameter tuning (learning rate, discount factor, network architecture, replay buffer size).
- 4.3 Monitor training curves (reward per episode, loss).

Phase 5: Evaluation and Comparison

5.1 Evaluate trained agents on held-out test traces.

5.2 Run baseline heuristics (FCFS, SJF, WFP, UNICEP) on the same traces.

5.3 Compute metrics: average bounded slowdown, average wait time, utilization, makespan.

5.4 Reproduce comparison tables and plots from the paper.

Datasets

Dataset / Source	Type	Location / Notes
Parallel Workloads Archive	HPC job traces	https://www.cs.huji.ac.il/labs/parallel/workload/
ANL Intrepid trace	Job trace	Available from above archive
SDSC Blue/SP2 trace	Job trace	Available from above archive
CQSim test traces	Bundled	Included in CQSim repository

Models, Tools, and Software

Tool / Library	Version	Purpose / URL
CQSim	latest	Cluster queue simulator https://github.com/NSAP/CQSim (or as referenced)
CQGym	latest	OpenAI Gym wrapper for CQSim Check paper’s supplementary or author’s GitHub
OpenAI Gym / Gymnasium	≥ 0.26	RL environment interface https://gymnasium.farama.org/
Stable-Baselines3	≥ 1.6	RL algorithm implementations (PPO, A2C, DQN) https://stable-baselines3.readthedocs.io/
PyTorch	≥ 1.10	Deep learning backend
NumPy	≥ 1.21	Numerical operations
Matplotlib / Seaborn	≥ 3.4	Performance comparison plots
Pandas	≥ 1.3	Job trace data manipulation

Hardware Requirements

- RL training: GPU recommended but not required ($\sim$ hours on CPU).
- RAM: ≥ 8 GB.